🛡️ Module 11: Reverse Engineering, Fuzzing and Binary Exploitation

```
# Exercise 16: Exploring Additional Capabilities of Address Sanitizer
```

Scenario

Advanced Address Sanitizer (ASan) Options allow penetration testers and binary auditors to uncover subtle memory lifecycle vulnerabilities—such as **Use-After-Return (UAR)** errors—that standard ASan configurations do not detect by default.

Key technical concepts of Use-After-Return detection:
1. **Scope and Lifetime Violations**: A stack variable pointer returned from a function becomes invalid after the function stack frame collapses. Dereferencing that pointer outside the function scope creates a Use-After-Return vulnerability.
2. **ASAN Runtime Environment Flags**: Standard compilation (`gcc -O -fsanitize=address file.c -o file`) catches out-of-bounds accesses, but requires explicit runtime activation of stack-use-after-return tracking via **ASAN_OPTIONS="detect_stack_use_after_return=1"**.
3. **Runtime Interception**: When enabled, ASan allocates fake stack frames in the heap to detect dereferences of expired stack addresses and reports a detailed `stack-use-after-return` error log pointing to the exact instruction offset.

In this exercise, we log into **Ubuntu 20.04 Fuzzing-AFL-ADV** VM (**student** / **studentpassword**), inspect sample C program **file.c**, compile with GCC Address Sanitizer support (`gcc -O -fsanitize=address file.c -o file`), execute with environment variable **env ASAN_OPTIONS="detect_stack_use_after_return=1" ./file**, and catch the Use-After-Return stack corruption vulnerability.

Objective

The objective of this lab is to help students learn how to:
- Authenticate to Ubuntu 20.04 Fuzzing-AFL-ADV target VM (**student** / **studentpassword**)
- Inspect source file **file.c** exhibiting Use-After-Return behavior
- Compile C code with optimization and Address Sanitizer enabled: **gcc -O -fsanitize=address file.c -o file**
- Execute binary with Use-After-Return flag enabled: **env ASAN_OPTIONS="detect_stack_use_after_return=1" ./file**
- Analyze ASan error reports detailing stack-use-after-return violations and memory addresses

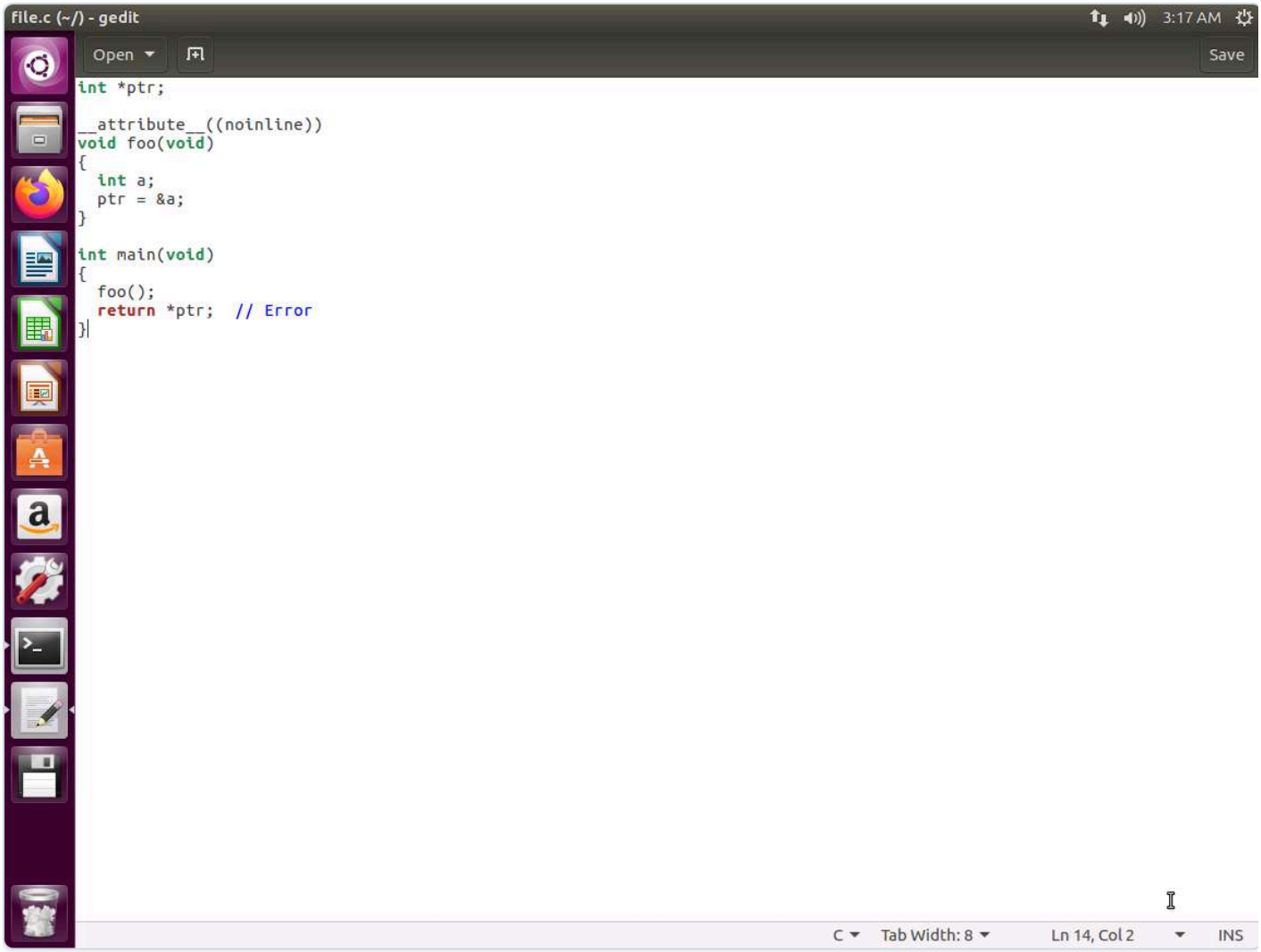
---

Task 1: Inspect file.c, Compile with ASan, Enable detect_stack_use_after_return=1, and Intercept Memory Errors

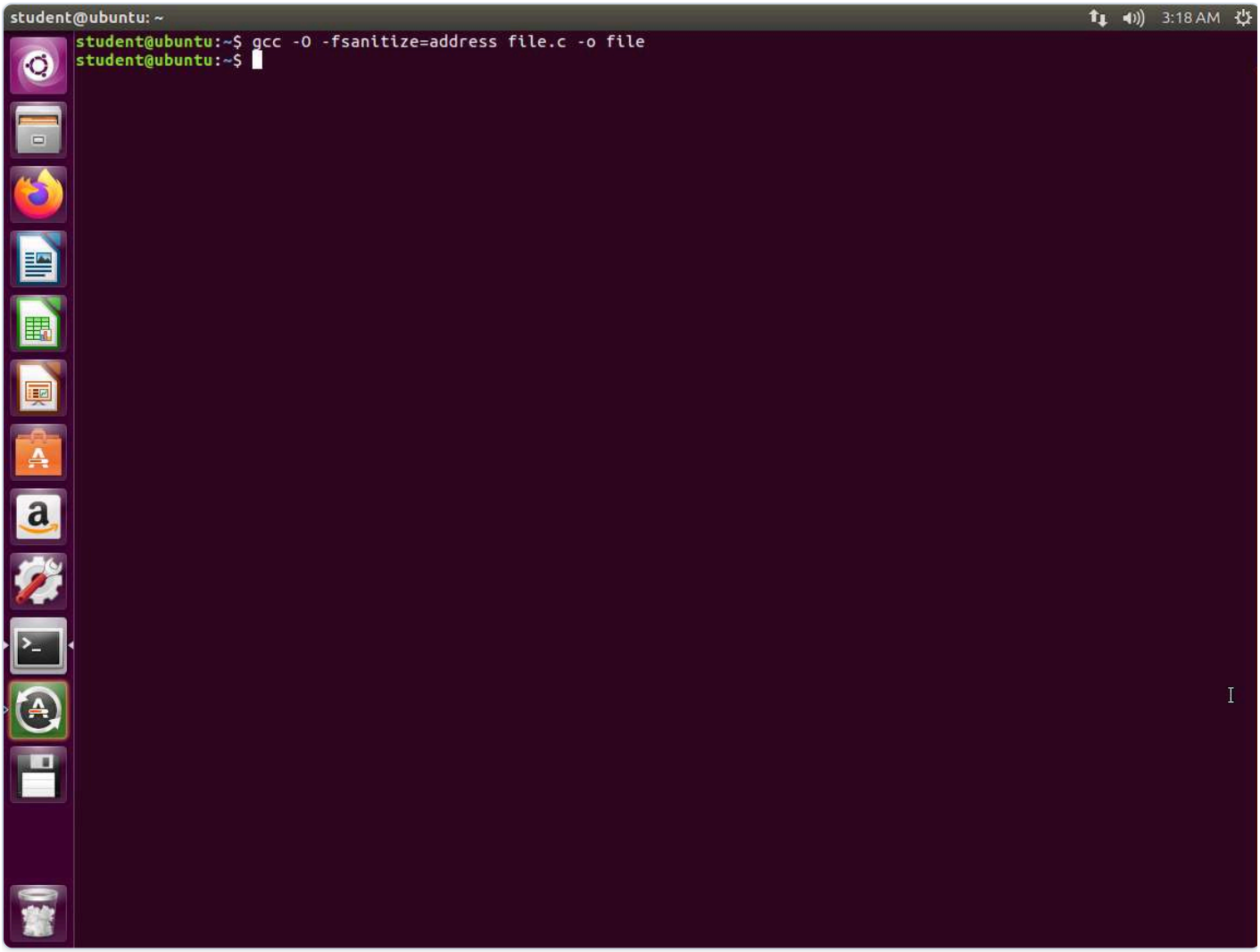
In this task, we inspect `file.c`, compile with `-fsanitize=address`, enable runtime UAR detection (`detect_stack_use_after_return=1`), and capture stack-use-after-return violations.

Lab Tasks

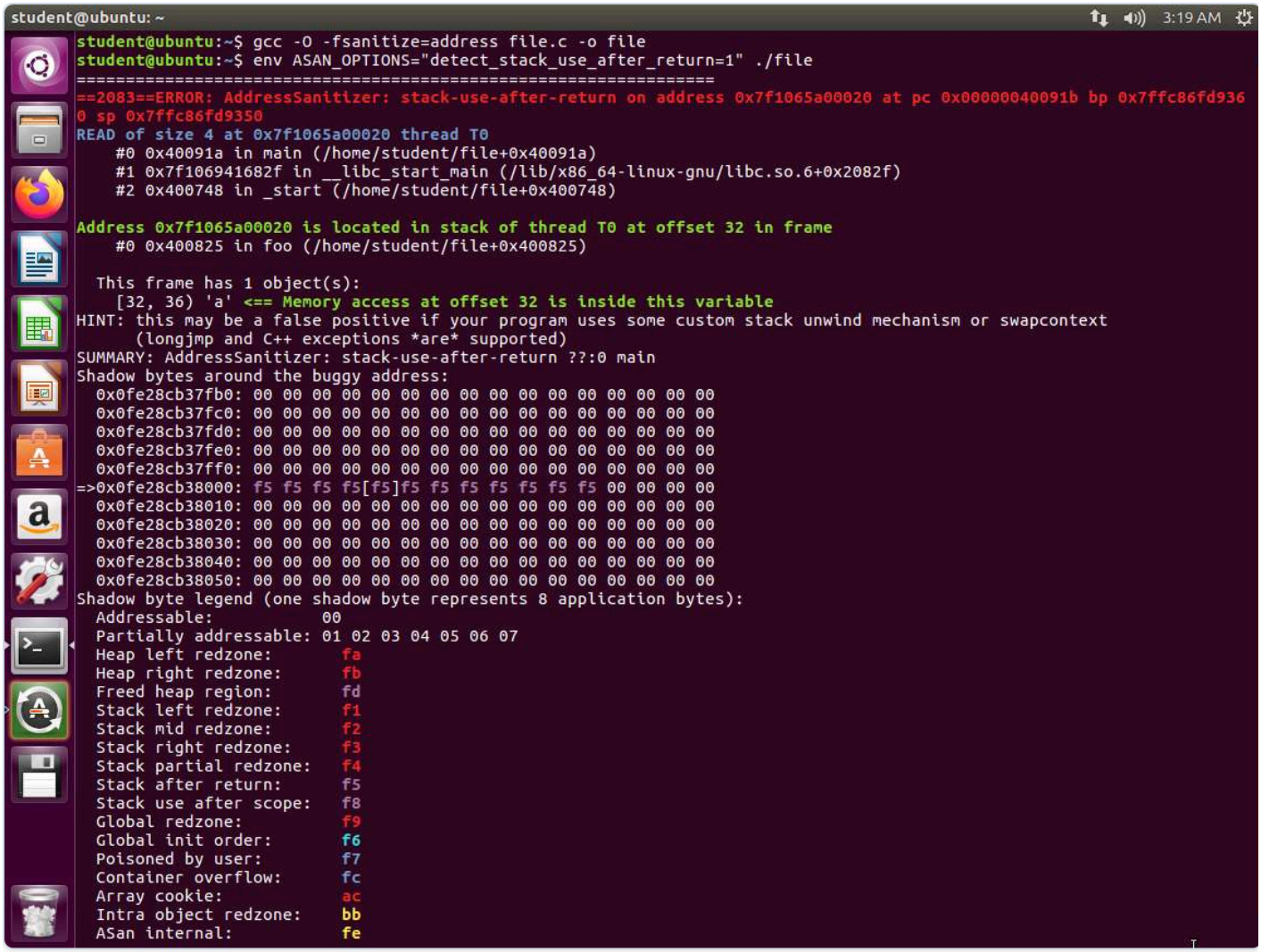
1. Log into **Ub20 Fuzzing-AFL-ADV** VM using credentials (**student** / **studentpassword**). Inspect source file **file.c**.



1. Compile source code using GCC with optimization and Address Sanitizer enabled:
   gcc -O -fsanitize=address file.c -o file



1. Execute binary passing `detect_stack_use_after_return=1` via `ASAN_OPTIONS` environment variable:
   `env ASAN_OPTIONS="detect_stack_use_after_return=1" ./file`
   Observe output: AddressSanitizer intercepts and logs a `stack-use-after-return` memory error.



This concludes the lab exercise.